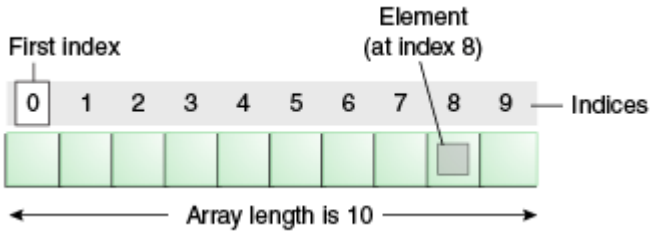
Tableaux dynamiques

Définition



Un tableau est une structure de données qui permet de stocker des éléments de même type dans une région contiguë de la mémoire. On peut accéder efficacement à un élément via son indice.

Un tableau dynamique est un tableau dont la taille peut varier au cours du temps.

En TypeScript, on utilise le type `Array` pour définir un tableau dynamique.

Remarque : en TypeScript, les tableaux sont **homogènes**, c'est-à-dire que tous les éléments du tableau sont du même type.

Déclaration d'un tableau

- Utilisation de crochets

```
let fruits: strings[] = ["Pomme", "Orange", "Banane"];
```

- Utilisation du type générique `Array`

```
let fruits: Array<strings> = ["Pomme", "Orange", "Banane"];
```

- **Remarque :** les tableaux en TypeScript sont **génériques**. Cela signifie que l'on peut définir le type des éléments du tableau. Ci-dessus, on définit un tableau de chaînes de caractères. On peut définir des tableaux de nombres, de booléens, d'objets, etc ...

- **Remarque :** il faut toujours affecter une valeur à un tableau avant de l'utiliser. Dans les exemples ci-dessus, on affecte à la variable un tableau littéral. On peut aussi utiliser l'opérateur `new` pour créer un tableau vide.

Opérateur `new`

- `new` permet d'allouer un tableau en mémoire (en fait création d'un nouvel **objet**)
- Création d'un tableau vide

```
let tab: Array<number> = new Array<number>();
```

- On peut aussi remplir des éléments à la déclaration

```
let jours: Array<string> = new Array<string>(
  "lundi",
  "mardi",
  "mercredi",
  "jeudi",
  "vendredi",
  "samedi",
  "dimanche"
);
```

- Création d'un tableau d'une taille donnée

```
let tab: Array<number> = new Array<number>(5);
```

Attention : la valeur des éléments est `undefined`, il faut les affecter avant de les utiliser

```
let tab: Array<number> = new Array<number>(5);
console.log(tab[0]); // undefined
tab[0] = 1;
console.log(tab[0]); // 1
```

```
undefined
1
```

Accès à un élément : opérateur `[]`

```
let tab: Array<number> = new Array<number>(1, 2, 3, 4);
```

- Taille d'un tableau : `tab.length`
- Numérotation des éléments de 0 à `tab.length - 1`
- Accès à l'élément d'indice `i` : `tab[i]`
- Par exemple, affichage des éléments dans un tableau :

```
for (let i = 0; i < tab.length; ++i) console.log("tab[" + i + "] = " +
tab[i]);
```

```
tab[0] = 1
tab[1] = 2
tab[2] = 3
tab[3] = 4
```

Validité des indices

Attention : TypeScript ne vérifie pas la validité d'un indice `i`

1. $0 \leq$ `i` $<$ `tab.length` : accès à l'élément

```
tab[2] = 42; // Modification de l'élément
let x = 2 * tab[2]; // Accès et utilisation de l'élément
```

2. $i < 0$: ne pas faire !

3. `i >= tab.length` :

    - modification de la taille du tableau tel que `tab.length` $=$ `i+1`
    - Les nouveaux éléments reçoivent la valeur `undefined`
    - Après l'instruction `tab[7] = 10`, le tableau contient :

```
[1, 2, 42, 4, undefined, undefined, undefined, 42];
```

Méthodes de la classe `Array`

Un tableau est un **objet**. Plusieurs méthodes sont disponibles pour manipuler les tableaux. Dans ce cours, on utilisera principalement les deux méthodes suivantes :

- `push` : ajoute un élément à la fin du tableau

```
let tab: Array<number> = new Array<number>();
tab.push(1);
tab.push(2);
tab.push(3);

console.log(tab);
```

```
[ 1, 2, 3 ]
```

- `pop` : supprime le dernier élément du tableau et retourne sa valeur

```
let tab: Array<number> = new Array<number>(1, 2, 3);
let x = tab.pop();

console.log(tab);
console.log(x);
```

```
[ 1, 2 ]
3
```

Construction d'un tableau élément par élément

Remarque : dans de nombreux cas, on définit un tableau vide que l'on remplira au fur et à mesure de l'exécution du programme.

Exemple : construction des 10 premiers nombres pairs

```
let nombres_pairs: Array<number> = new Array<number>(); // Création d'un
tableau vide

for (let i = 0; i < 10; ++i) nombres_pairs.push(2 * i);
```

Parcourir un tableau

- On peut parcourir un tableau avec une boucle `for`

```
let tab: Array<number> = new Array<number>(1, 2, 3, 4);

for (let i = 0; i < tab.length; ++i) console.log(tab[i]);
```

```
1
2
3
4
```

- On peut aussi utiliser une boucle `for ... of` : itérations sur les éléments

```
let tab: Array<number> = new Array<number>(1, 2, 3, 4);

for (let valeur of tab) console.log(valeur);
```

```
1
2
3
4
```

- Ou une boucle `for ... in` : itérations sur les indices

```
let tab: Array<number> = new Array<number>(1, 2, 3, 4);

for (let i in tab) console.log(tab[i]);
```

```
1
2
3
4
```

Attention : dans ce cas, le type de la variable `i` est `string` et non `number`

Remarque : quand on parcourt un tableau, on peut utiliser n'importe quelle boucle. Cependant, la boucle `for ... of` est la plus adaptée si on n'a pas besoin de connaître l'indice des éléments.

Affectation entre tableaux

Considérons les instructions suivantes :

```
let t1: Array<number> = [1, 2, 3, 4, 5];
let t2: Array<number> = t1;

console.log(t1);
console.log(t2);
```

```
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 3, 4, 5 ]
```

Question : Quel est le rapport entre `t1` et `t2` ?

1. `t1` et `t2` font référence à deux objets tableaux distincts qui ont la même valeur (la séquence `1, 2, 3, 4, 5`)

   ou

2. `t1` et `t2` font référence au même objet tableau, ce qui signifie que modifier `t1` entraîne une modification de `t2` et vice versa.

Essai

```
let t1 = [1, 2, 3, 4, 5];
let t2 = t1;

t2[2] = 42;

console.log(t1);
console.log(t2);
```

```
[ 1, 2, 42, 4, 5 ]
[ 1, 2, 42, 4, 5 ]
```

Réponse : on est donc dans le second cas

Passage en paramètre d'une fonction

Conséquence : quand on passe un tableau en paramètre d'une fonction, on passe en fait une référence vers le tableau et non une copie du tableau. Cela signifie que si la fonction modifie le tableau, cette modification est visible dans le programme appelant.

Exemple : fonction qui ajoute un élément à un tableau

```
function ajoute(tab: Array<number>, x: number): void {
  tab.push(x);
}

let t: Array<number> = [1, 2, 3, 4, 5];

ajoute(t, 6);

console.log(t);
```

```
[ 1, 2, 3, 4, 5, 6 ]
```

Qu'est-ce que l'efficacité ?

On définit dans ce cours l'efficacité comme le temps d'exécution d'un algorithme. Selon la nature de l'algorithme, on peut exprimer ce temps en nombre d'opérations, d'instruction, d'itérations ou en temps d'exécution ... Pour un même algorithme, le choix des structures de données employées peut avoir un

impact important sur l'efficacité. Dans ce cours, on va explorer trois structures de données pour stocker des éléments :

- les tableaux dynamiques
- les listes chaînées
- les tables de hachage

Pour chacune de ces structures, on va étudier l'efficacité des opérations suivantes :

- accès à un élément
- ajout d'un élément
- suppression d'un élément
- recherche d'un élément

On discutera aussi de leur intérêt et de leur inconvénient d'un point de vue gestion de la mémoire.

Efficacité des tableaux

Implémentation d'un tableau dynamique

Le langage TypeScript possède la structure `Array` qui permet de manipuler des tableaux dynamiques de manière transparente sans se soucier de les redimensionner si besoin. Ce n'est pas le cas de tous les langages de programmation. Dans ce cours, on va donc définir une structure de données `Tableau` qui permet de manipuler des tableaux dynamiques. Cela nous permettra aussi de comprendre comment fonctionne un tableau dynamique et d'analyser le coût des différentes opérations.

Pour représenter un tableau dynamique, on a besoin d'avoir un tableau alloué en mémoire et de connaître le nombre effectif d'éléments dans le tableau. On va donc définir une classe `Tableau` qui contient deux attributs :

```
class Tableau {
  private _taille: number;
  private _tab: Array<number>;

  constructor(capacité = 100) {
    this._taille = 0;
    this._tab = new Array(capacité);
  }

  get taille() {
    return this._taille;
  }
}
```

Remarque : cette structure est artificielle car la classe `Array` offre déjà toutes les fonctionnalités nécessaires pour manipuler un tableau dynamique. Mais si on était en C, définir une telle structure aurait un sens.

Accès à un élément

L'accès à un élément dans un tableau est très efficace car on y accède directement via son indice (opération qui est optimisée par le compilateur). On dit que l'accès à un élément est en **temps constant**.

Dans notre code, cela revient à ajouter une méthode `élément` à la classe `Tableau` :

```
class Tableau {
  // ...

  public élément(i: number): number {
    return this._tab[i];
  }
}
```

Ajout d'un élément à la fin du tableau

Deux cas peuvent se présenter :

1. Le tableau n'est pas plein : on ajoute l'élément à l'indice `taille` et on incrémente `taille` de 1 ; cela se fait en temps constant ;
2. Le tableau est plein : on crée un nouveau tableau de taille strictement plus grand que `taille` et on copie les éléments du tableau initial dans le nouveau tableau. On ajoute ensuite l'élément à l'indice `taille` et on incrémente `taille` de 1 ; cela se fait en temps linéaire $O(taille)$. On ne se contente pas d'incrémenter `taille` de 1 car on veut pouvoir ajouter un nouvel élément sans avoir à redimensionner le tableau à chaque fois. **Remarque :** dans de nombreux langages, cette opération est faite au niveau du compilateur et optimisée pour être aussi efficace que dans le premier cas, c'est-à-dire en temps constant $O(1)$ (on dit qu'il est en temps constant amorti).

```
class Tableau {
  // ...

  public ajouteFin(x: number): void {
    if (this._taille < this._tab.length) {
      this._tab[this._taille] = x;
      this._taille++;
    } else {
      const nouveau_tab: Array<number> = new Array<number>(
        2 * this._tab.length,
      );
      for (let i = 0; i < this._taille; ++i) nouveau_tab[i] =
this._tab[i];
      nouveau_tab[this._taille] = x;
      this._tab = nouveau_tab;
      this._taille++;
    }
  }
}
```

Ajout d'un élément à un indice donné

La manière la plus simple est d'ajouter l'élément à la fin en temps constant puis de le faire *remonter* jusqu'à l'indice voulu. Le coût de cette *remontée* est égale au nombre de décalages effectués. Dans le pire des cas quand on souhaite insérer à l'indice 0, on doit décaler tous les éléments du tableau. On dit que l'opération est en **temps linéaire**.

```
class Tableau {
  // ...

  public ajouteIndice(i: number, x: number): void {
    this.ajouteFin(x);
    for (let j = this._taille - 1; j > i; --j) {
      const tmp = this._tab[j];
      this._tab[j] = this._tab[j - 1];
      this._tab[j - 1] = tmp;
    }
  }
```

Suppression d'un élément à la fin du tableau

On a juste à décrémenté `taille` de 1. Cela se fait en temps constant.

```
class Tableau {
  // ...

  public supprimeFin(): void {
    this._taille--;
  }
}
```

Suppression d'un élément à un indice donné

On a juste à faire descendre les éléments du tableau à partir de l'indice `i` jusqu'à la fin. Le coût de cette *descente* est égale au nombre de décalages effectués. Dans le pire des cas quand on souhaite supprimer l'élément à l'indice 0, on doit décaler tous les éléments du tableau. On dit que l'opération est en **temps linéaire**. Il faut aussi décrémenter `taille` de 1.

```
class Tableau {
  // ...

  public supprimeIndice(i: number): void {
    for (let j = i; j < this._taille - 1; ++j) {
      const tmp = this._tab[j];
      this._tab[j] = this._tab[j + 1];
      this._tab[j + 1] = tmp;
    }
    this._taille--;
```

```
  }
}
```

Recherche d'un élément

On parcourt le tableau jusqu'à trouver l'élément recherché. Le coût de cette recherche est égale au nombre de comparaisons effectuées. Dans le pire des cas quand on ne trouve pas l'élément, on doit comparer tous les éléments du tableau. On dit que l'opération est en **temps linéaire**.

```
class Tableau {
  // ...

  public recherche(x: number): boolean {
    for (let e of this._tab) if (e === x) return true;
    return false;
  }
}
```

Remarque : rechercher le minimum ou le maximum d'un tableau se fait en temps linéaire aussi.

Remarque : si le tableau est trié, la recherche peut se faire de manière beaucoup plus efficace mais il y a un coût à payer pour trier le tableau (ou le maintenir trié si on le modifie)

Récapitulatif de notre classe `Tableau`

```
class Tableau {
  private _taille: number;
  private _tab: Array<number>;

  constructor(capacité = 100) {
    this._taille = 0;
    this._tab = new Array(capacité);
  }

  get taille() {
    return this._taille;
  }

  public élément(i: number): number {
    return this._tab[i];
  }

  public ajouteFin(x: number): void {
    if (this._taille < this._tab.length) {
      this._tab[this._taille] = x;
      this._taille++;
    } else {
      const nouveau_tab: Array<number> = new Array<number>(
        2 * this._tab.length,
```

```
      );
      for (let i = 0; i < this._taille; ++i) nouveau_tab[i] =
this._tab[i];
      nouveau_tab[this._taille] = x;
      this._tab = nouveau_tab;
      this._taille++;
    }
  }

  public ajouteIndice(i: number, x: number): void {
    this.ajouteFin(x);
    for (let j = this._taille - 1; j > i; --j) {
      const tmp = this._tab[j];
      this._tab[j] = this._tab[j - 1];
      this._tab[j - 1] = tmp;
    }
  }

  public supprimeFin(): void {
    this._taille--;
  }

  public supprimeIndice(i: number): void {
    for (let j = i; j < this._taille - 1; ++j) {
      const tmp = this._tab[j];
      this._tab[j] = this._tab[j + 1];
      this._tab[j + 1] = tmp;
    }
    this._taille--;
  }

  public recherche(x: number): boolean {
    for (const e of this._tab) if (e === x) return true;
    return false;
  }
}
```

Que retenir ?

Coût

Le coût d'une opération sur une structure de données n'est pas toujours la même. On a vu ici deux coûts qui sont par ordre croissant :

- **Temps (ou coût) constant :** accès à un élément, ajout / suppression d'un élément à la fin du tableau
- **Temps linéaire :** ajout / suppression d'un élément à un indice donné, recherche

Tableaux dynamiques

- efficacité en termes de gestion de la mémoire (zone contiguë en mémoire)
- efficacité pour l'accès à un élément, l'ajout / suppression d'un élément à la fin du tableau
- inefficace pour l'ajout / suppression d'un élément à un indice donné, la recherche

- la recherche peut être améliorée si le tableau est trié (mais cela a un coût)